

Understanding Big- O Notation

Intuition Behind Big- O

Big- O notation provides a way to describe the growth of a function as the input size n becomes very large. The idea is that for analyzing algorithms, constant factors and lower-order terms matter less than the *rate of growth* of the dominant term.

- For example, $3n + 5$ grows roughly like n for large n . The constant multiplier (3) and the constant additive term (5) become negligible in comparison to the linear term n .
- Similarly, $7n^2 + 2n + 8$ grows like n^2 . The quadratic term dominates the linear and constant terms as n increases.
- In general, when we write $f(n) \in O(g(n))$, we mean that $f(n)$ is *bounded above* by some constant multiple of $g(n)$ when n is sufficiently large.

Formally:

$$f(n) \in O(g(n)) \iff \exists c > 0, n_0 \geq 1 : f(n) \leq c \cdot g(n) \quad \forall n \geq n_0.$$

This definition captures the idea that $g(n)$ represents the “shape” or dominant growth of $f(n)$, up to constant scaling.

Practice Problems

Show that each of the following statements is true using the definition of Big- O :

1. $5n^3 + 4n^2 + 6 \in O(n^3)$
2. $20n + 15 \log n \in O(n)$
3. $50n \log n + 7n^2 \in O(n^2)$
4. $12 \log^2 n + 100 \log n \in O(\log^2 n)$
5. $9n^4 + 3n^3 + 2n^2 + 1 \in O(n^4)$

Key Insight

To solve problems like these, always ask:

- What is the *highest-order term* in the function as $n \rightarrow \infty$?
- Can I find a constant c such that the entire expression is less than or equal to c times the dominant term for large n ?

—